

Course-Grounded Study Assistant

An Android app that answers student questions using only the material their teacher has approved.

Course	CSE 346 — Information System Design & Software Engineering Lab
Institution	Southeast University
Platform	Native Android (phone)
Language	Java + XML layouts
Size	38 Java files, ~5,700 lines, 26 screens

1. Why I built this

Students use ChatGPT to study, but it answers from the whole internet. You cannot tell whether an answer matches your syllabus, and it will confidently invent things your teacher never taught. Google Classroom has the opposite problem: it stores your files safely but cannot answer a single question about them.

My app closes that gap. The teacher uploads slides, notes and PDFs and approves them. The AI then answers **only** from that approved material, and shows which file each answer came from.

The one-line pitch

Google Classroom stores your course material. ChatGPT answers questions but not from your course. This app does both — and the teacher controls the source.

2. Technology used

Layer	What I used	Why
Language	Java 17	Taught in our lab; no Kotlin needed
UI	XML layouts + Material Components	Standard Android, easy to show in class
IDE / Build	Android Studio, Gradle (Kotlin DSL)	Standard Android toolchain
Database	SQLite via SQLiteOpenHelper	Built in, works fully offline
PDF text	pdfbox-android	Extracts text from PDF uploads
PDF viewing	Android PdfRenderer	Built in — no extra library
PPTX / DOCX	java.util.zip (built in)	Both formats are ZIP files of XML
Network	OkHttp	One small POST; Retrofit would be overkill
AI model	Google Gemini (gemini-flash-lite-latest)	Free tier, OpenAI-compatible API

Min Android 7.0 (API 24), target API 37. Everything except the AI call runs offline.

3. How the project is organised

The code is split into four packages. This is the first thing to show on screen.

Package	Job	Main files
model	Plain data classes	User, Course, Material, Row
data	Storage and file reading	DatabaseHelper, DocumentImporter, FileStore, SessionManager, SeedData
engine	The AI brain	RetrievalEngine, OpenAiClient, AnswerResult
ui	All 26 screens	BaseActivity + one Activity per screen, Anim, CourseArt

The trick that keeps it small: BaseActivity

Every screen extends **BaseActivity**. It holds the things all screens need, so no screen repeats them: the database, the login session, the app bar, bottom navigation, dark/light theme, English/Bangla language, page animations, and the floating “Ask AI” button.

Example to show the teacher — the AI button is added once, in BaseActivity, by overriding setContentView. That is why it appears on every student screen without editing 26 layout files:

```
@Override
public void setContentView(int layoutResID) {
    super.setContentView(layoutResID);
    attachAiBubble(); // adds the Ask AI button
}
```

4. The database

One SQLite file with 8 tables:

Table	Holds
users	Students and teachers, with role and plan
courses	Course code, title, faculty, join code
enrollments	Which student is in which course
materials	Uploaded slides/notes + the original file path
chunks	Material split into small searchable passages
questions	Every question asked (for the Progress screen)
bookmarks	Saved materials
notifications	Alerts when material is added or approved

The **chunks** table is the important one. It is my stand-in for a vector database — one row per searchable passage.

4. The core feature: RAG

RAG means **R**etrieval **A**ugmented **G**eneration. Instead of trusting the AI's memory, I first find the right passage from the teacher's material, then hand it to the AI and ask it to answer using that. Three steps:

Step	What happens	Where in my code
1. Retrieve	Find the best matching passages from approved material only	RetrievalEngine.java
2. Augment	Put those passages into the prompt	OpenAIClient.buildRequest()
3. Generate	Gemini writes the answer from them	OpenAIClient.ask()

Step 1 in detail — how I find the right passage

I use **TF-IDF**, a classic information-retrieval formula. It scores a passage higher when it contains the question's rare words:

```
idf(t) = ln(1 + N / df(t))
tf(t) = 1 + ln(count of t in passage)
score = SUM( tf x idf ) / sqrt(passage length) x typeWeight
```

Dividing by *sqrt(length)* stops long passages winning just for being long. *typeWeight* prefers a lecture slide (1.00) over a quiz (0.60), because a quiz only mentions a term while a lecture explains it.

I also compute **coverage** — how many words of the question were actually found. If coverage is below **34%**, the course clearly does not cover the question, so no course material is sent and the AI answers from general knowledge instead. The screen says which of the two happened.

How a file becomes searchable

Stage	What my code does
1. Read the file	DocumentImporter extracts text: PDF via pdfbox, PPTX/DOCX by unzipping the XML
2. Clean it	Joins broken lines, decodes & and — entities
3. Split it	insertChunks() cuts it into ~55-word passages, breaking at sentence or slide boundaries
4. Save	One row per passage in the chunks table

A real bug I found and fixed

Lecture slide bullets usually have no full stop. My first chunker only split on “. ! ?”, so a whole 310-word deck became ONE giant passage and scoring became meaningless. I added a paragraph-break rule. The same deck now produces 5 clean passages, one per slide boundary.

5. My unique feature: the Content Lock

This is the part to spend the most time on. It is what makes the app different from every other AI study tool.

Uploaded material is invisible until the teacher presses **Approve and Lock**. It is enforced in the database query itself, not in the UI — so there is no way around it:

```
SELECT ch.text FROM chunks ch
  JOIN materials m ON m.id = ch.material_id
 WHERE ch.course_id = ? AND m.approved = 1
```

That single **AND m.approved = 1** is the whole security model. If a passage is not approved it is never loaded, so it can never be sent to the AI.

If you remember one thing

Every other AI tool answers from the internet. Mine answers from what the teacher approved — enforced by that one SQL condition — and every answer names the file it came from, so the student can check it.

The live demo to run for your teacher

Do this	They will see
1. Log in as teacher, upload a PDF, tick Approve and Lock	Material goes live
2. Log in as student, tap Ask AI, ask about that file	Correct answer, 100% coverage, and the file name cited
3. Back as teacher, press Revoke	Confirmation dialog explains the consequence
4. As student, ask the same question	The AI no longer uses it — the lock works instantly

6. All features

Student	Teacher
Join a course with a code	Create courses, auto join codes
Ask AI — floating button on every screen	Upload PDF / PPTX / DOCX / TXT / MD / CSV
Read uploaded PDFs as real pages	Approve and Lock, or Revoke
Answers cite their source file	See pending-review badges
Search all approved material	Student counts and analytics
Bookmarks with swipe and undo	Notifications to enrolled students
Progress: grounding rate, per course	
Light/dark theme, English/Bangla	

7. Code tour — what each key file does

If the teacher asks “show me the code”, open these in this order.

File	What to say about it
BaseActivity.java	“Every screen extends this. It gives all 26 screens the database, session, theme, language, and the Ask AI button — written once, not 26 times.”
DatabaseHelper.java	“All 8 tables and every query. insertChunks() is where an uploaded file gets cut into searchable passages.”
DocumentImporter.java	“Turns any upload into plain text. PPTX and DOCX are ZIP files of XML, so I unzip them with java.util.zip — no library needed.”
RetrievalEngine.java	“The TF-IDF search. loadApprovedPassages() has the AND approved = 1 that enforces the Content Lock.”
OpenAiClient.java	“The single network call. It builds the prompt with the retrieved passages and maps every failure to a clear on-screen message.”
ChatActivity.java	“The Ask AI chat. It searches across ALL my courses at once and remembers the last 6 turns, so follow-up questions work.”
PdfViewActivity.java	“Reads the original PDF using Android's built-in PdfRenderer. Pages render as you scroll and are cached.”

8. Problems I solved (good viva answers)

Problem	My fix
Answers repeated themselves	Made chunks non-overlapping and skip passages that restate one already used
Answers started mid-sentence	Chunk only at sentence boundaries
A quiz outranked the lecture	Added typeWeight so lectures score higher
A whole slide deck became one chunk	Also split on paragraph/slide breaks
A Word file made a 41,000-character chunk	Kept Word paragraphs as real paragraphs, plus a hard size cap
App logged itself out	A database upgrade dropped the users table, so the saved user id pointed at nothing. Now the upgrade clears the session properly
Students only saw stripped text	Now the original file is stored too, and opens as a real PDF

9. How to run it

1. Open the **StudyAssistant** folder in Android Studio and let Gradle sync.
2. Put your AI key in **local.properties**: OPENAI_API_KEY=...
3. Connect a phone with USB debugging on, or start an emulator.
4. Press **Run**.

Demo logins — teacher@seu.edu.bd / teacher123 • student@seu.edu.bd / student123

The API key is read from local.properties into BuildConfig at build time, so it is never written in the source code and never committed.